

Long Context and Faithful Generation in Large Language Models

From position geometry to evidence-grounded selective generation

What this lecture is about

How large language models handle very long inputs, why they still fail on evidence use and citation, and why robust systems usually combine *inside-model long-context extension* with *outside-model evidence control*.

Running example

Task

A user asks:

“According to a 300-page policy manual and its appendices, which rules govern incident reporting, who signs off, and which exceptions apply? Please provide sentence-level citations.”

- The answer may depend on a few short clauses hidden inside a very long document.
- Some evidence may be in the middle; some may be in appendices.
- The system must do more than sound plausible: it must point to the right support.

Bottom line: This one task exposes long-context use, retrieval quality, hallucination, and citation quality at once.

Four failure modes

Inside the prompt

- **Long-context failure:** the right paragraph is present but ignored.
- **Citation failure:** the answer is plausible or even correct, but the cited sentence does not support it.

Outside the prompt

- **Retrieval failure:** the relevant appendix never reaches the model.
- **Hallucination:** the model adds unsupported or contradicted content.

Important distinction

Retrieval failure and hallucination often co-occur, but they are not the same error. One is an evidence-selection problem; the other is an answer-faithfulness problem.

Two frameworks for the lecture

Framework A: position-geometry preservation

- Focus: **long-context stability**
- Main question: can attention stay well-behaved as positions grow far beyond training length?
- Representative methods: PI, YaRN, LongRoPE, LongRoPE2

Framework B: evidence-grounded selective generation

- Focus: **faithful answering**
- Main question: can the system select, verify, and cite evidence rather than free-generate unsupported text?
- Representative methods: Self-RAG, CRAG, GraphRAG, LongCite, SelfCite

Practical framing

Framework A is mainly about *inside-model use of long positions*. Framework B is mainly about *outside-model and around-model control of evidence*.

From text to token vectors

- A language model does not directly read words; it reads a sequence of **tokens**.
- A token may be a whole word, a subword piece, or punctuation.
- Each token ID is mapped to a learned vector called an **embedding**.

Embedding lookup

$$E \in \mathbb{R}^{|V| \times d}, \quad x_t = E[w_t] \in \mathbb{R}^d$$

Symbols: $|V|$ = vocabulary size; d = hidden dimension; w_t = token ID at position t ; x_t = embedding vector.

Interpretation: The model begins with vectors $x_1, \dots, x_n$, not with raw text strings.

Why it matters: Attention, retrieval matching, and position handling all operate on these vectors.

Pitfall

Tokens are *not* the same as words. A long document can contain far more tokens than its page count suggests.

What the model predicts

- A causal language model predicts the **next token** given the prefix.
- Complex behavior emerges from repeating this simple objective at scale.

Autoregressive modeling

$$p_{\theta}(w_{1:n}) = \prod_{t=1}^n p_{\theta}(w_t \mid w_{<t}), \quad \mathcal{L}_{\text{LM}}(\theta) = -\mathbb{E}_{w_{1:n} \sim \mathcal{T}} \sum_{t=1}^n \log p_{\theta}(w_t \mid w_{<t})$$

Symbols: $w_{1:n}$ = token sequence; $w_{<t}$ = prefix before position t ; θ = model parameters; $\mathcal{T}$ = training data distribution.

Interpretation: The model is trained to assign high probability to the true next token at every position.

Why it matters: Long-context ability and grounded generation both inherit the strengths and weaknesses of this next-token objective.

Bottom line: Everything later in the lecture changes the information available to this next-token predictor.

Training versus inference and the causal mask

- **Teacher forcing:** during training, position t sees the true prefix $w_{<t}$.
- During generation, the model conditions on its own previously generated tokens instead.

Causal mask

$$M_{ij} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

Symbols: M_{ij} = mask entry for query position i and key position j .

Interpretation: Future tokens are blocked, so the model cannot “peek ahead” during training or inference.

Why it matters: This keeps autoregressive prediction honest and explains why generation is sequential even when training can be parallelized.

Why attention exists

- A token should not average all earlier tokens equally.
- It needs a **data-dependent weighted average**: different queries focus on different earlier content.

Scaled dot-product attention

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} + M_{ij}, \quad \alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{m \leq i} \exp(s_{im})}, \quad \hat{h}_i = \sum_{j \leq i} \alpha_{ij} v_j$$

Symbols: q_i, k_j, v_j = query, key, value vectors; d_k = key dimension; M_{ij} = causal mask entry.

Interpretation: The query says what the current position needs, keys say what earlier positions offer, and values are the information copied forward.

Why it matters: Long-context failure often appears as bad attention allocation: the model attends to the wrong places or fails to preserve useful signal at long distances.

Attention as selective focus

Tiny numerical example

Suppose the current token is `signs` and the model scores three earlier tokens:

$$(s_{\text{manager}}, s_{\text{incident}}, s_{\text{the}}) = (2, 1, 0)$$

Softmax gives roughly

$$\begin{aligned} (\alpha_{\text{manager}}, \alpha_{\text{incident}}, \alpha_{\text{the}}) \\ \approx (0.665, 0.245, 0.090) \end{aligned}$$

How to read this

- Most weight goes to `manager`.
- Some weight goes to `incident`.
- Little weight goes to the uninformative token `the`.

Core intuition

Attention is learned routing over past tokens, not a fixed average.

Multi-head attention and one decoder block

Multi-head attention

$$\text{head}^{(m)} = \text{Attn}(XW_Q^{(m)}, XW_K^{(m)}, XW_V^{(m)}), \quad \text{MHA}(X) = \text{Concat}(\text{head}^{(1)}, \dots, \text{head}^{(N_h)})W_O$$

Symbols: $X \in \mathbb{R}^{L \times d}$ = hidden states for all L positions; N_h = number of heads; W_O = output projection.

Interpretation: Different heads can specialize in different relationships: local syntax, long-range reference, instruction following, or evidence lookup.

Why it matters: Long-context methods still operate inside this block, so students need a compact model of the actual decoder layer.

One decoder block

$$\tilde{X} = X + \text{MHA}(\text{LN}(X)), \quad X' = \tilde{X} + \text{MLP}(\text{LN}(\tilde{X}))$$

Symbols: LN = layer normalization; MLP = position-wise feed-forward network.

Interpretation: Attention mixes information *across positions*; the MLP transforms information *within each position*; residual paths keep earlier information available.

Why it matters: This is the minimal computational unit later modified by sparse attention, memory, and long-context adaptation methods.

Why positions matter

- Without position, the model cannot distinguish “dog bites man” from “man bites dog” when the token multiset is the same.
- **Context window:** the maximum number of tokens the model can process at once.
- These are different issues: the context window says *how much* fits; positional handling says *how order affects attention*.

Two broad design choices

- ① **Additive position encoding:** use $h_i = x_i + p_i$.
- ② **Relative-position scoring:** change the attention score so it depends on distance or phase, as in ALiBi or RoPE.

Critical distinction

A model may accept a 128k-token prompt and still use it poorly. **Nominal context length** is an engineering property; **effective long-context skill** is a behavioral property.

Bottom line: Long-context work must solve both token capacity and stable use of positions inside that capacity.

Long window versus retrieval

Long context window

- Reads more tokens directly.
- Best when evidence already sits in one source.
- Failure mode: the model may still miss the right location.

Retrieval

- Searches a larger corpus and keeps a small evidence set.
- Best when only a few snippets matter.
- Failure mode: the key snippet may never be retrieved.

Retriever score

$$r(x, d) = u(x)^\top v(d), \quad z_{1:k} = \text{TopK}_{d \in \mathcal{D}} r(x, d)$$

Symbols: x = query; d = document or passage; u, v = learned embedding functions; $z_{1:k}$ = top- k retrieved items.

Interpretation: Dense retrieval scores query–document similarity in embedding space; lexical retrieval uses overlap statistics instead.

Why it matters: Later Framework B methods improve the full evidence-selection pipeline, not only answer generation.

Bottom line: Long windows expose more raw context; retrieval selects the most relevant evidence from a much larger space.

A toy retrieval pipeline

Question

Who signs off incident reports?

ID	Passage summary	Initial score
d_2	Incident workflow; states that the line manager signs.	0.91
d_5	Emergency-case exceptions.	0.77
d_4	Historical policy no longer in force.	0.53

- Initial retrieval is high-recall: it keeps d_2 , d_5 , d_4 .
- A reranker improves precision: it demotes obsolete d_4 .
- Packing keeps d_2 and d_5 because together they cover the rule and the exceptions.

Bottom line: Retrieval quality depends on three stages: recall, precision, and budget-aware packing.

Hallucination, support, and citation

- **Hallucination:** a generated claim is unsupported, contradicted, or incorrectly fused from multiple sources.
- Useful recurring grounded-QA error types: **unsupported insertion**, **contradiction**, **conflation**, and **citation error**.
- **Retrieval failure** is upstream: the relevant passage may never have reached the model.

Simple faithfulness score

$$\text{supp}(c, E) = \begin{cases} 1, & \text{if evidence } E \text{ supports claim } c \\ 0, & \text{otherwise} \end{cases}, \quad F(y, E) = \frac{1}{m} \sum_{j=1}^m \text{supp}(c_j, E)$$

Symbols: c_j = atomic claim in answer y ; E = selected evidence; m = number of claims.

Interpretation: Faithfulness is higher when more answer claims are directly supported by evidence.

Why it matters: Claim-level support reveals errors that whole-answer accuracy can hide.

Why claim-level scoring matters

A mostly correct answer can still hide one unsupported clause or one wrong citation span.

Primer checkpoint

You should now be able to explain

- ① what a token is and why tokens are not words,
- ② what next-token prediction means,
- ③ why attention uses queries, keys, and values,
- ④ why positional information is necessary,
- ⑤ why long context and retrieval are different tools,
- ⑥ why hallucination is not the same as retrieval failure,
- ⑦ why citation is stricter than a plausible answer.

Bottom line: The rest of the lecture builds on these mechanics rather than re-defining them.

Formal lens 1: long-context stability

- We care not only about input length ℓ but also about the position π of decisive evidence.
- A stable model should not collapse merely because evidence moves deeper into the document.

Stability gap

$$\Delta_{\text{LC}}(f; \ell, \pi) = \mathcal{M}(f; \ell_0, \pi_0) - \mathcal{M}(f; \ell, \pi)$$

Symbols: f = model; $\mathcal{M}$ = task metric; (ℓ_0, π_0) = easy baseline regime such as short context with answer near the end.

Interpretation: The gap measures how much performance degrades as contexts get longer or evidence moves to harder positions.

Why it matters: This makes “effective context size” measurable rather than marketing language.

Running example

A model can accept the full manual but still fail when the answer-bearing clause sits in the middle appendix pages.

Claim support and citation risk

One answer can mix supported and unsupported claims. Citation is stricter because the attached span must be the *right* support, not just any support.

Hallucination risk

$$R_{\text{hall}}(f) = \mathbb{E}_{(x,E)} \left[\frac{1}{|\mathcal{C}(y)|} \sum_{c \in \mathcal{C}(y)} (1 - \text{supp}(c, E)) \right]$$

Read it as: each claim is checked against the full evidence set E , where $y = f(x, E)$.

Citation risk

$$R_{\text{cite}}(f) = \mathbb{E}_{(x,E)} \left[\frac{1}{|\mathcal{C}(y)|} \sum_{c \in \mathcal{C}(y)} (1 - \text{supp}(c, z(c))) \right]$$

Read it as: each claim is checked only against the selected support span $z(c)$.

Difference that matters

Symbols: x = query; E = available evidence; $\mathcal{C}(y)$ = atomic claims in answer y ; $z(c)$ = span selected as support for claim c . **Interpretation:** R_{hall} penalizes unsupported claims; R_{cite} is stricter because the selected citation span must itself support the claim. **Why it matters:** Grounded long-form QA therefore needs claim-level evaluation and sentence-level attribution, not only answer-level accuracy. **Citation ideal:** a good span is **sufficient** enough to support the claim by itself and **necessary** enough that removing it weakens the claim.

Where systems fail

1. Inside-model

- poor position handling
- bad long-range attention allocation
- unstable long-distance memory

2. Evidence selection

- retrieval misses the key passage
- reranking promotes distractors
- compression drops exceptions

3. Answering

- unsupported insertion
- overgeneralization
- wrong or incomplete citation

Methods align with failure locations

PI / YaRN / LongRoPE mainly target **inside-model** failures; RAG / FiD / GraphRAG target **evidence selection**; Self-RAG / CRAG / LongCite / SelfCite target **answering and attribution**.

Benchmarks answer different questions

Benchmark family	Main question	Best use
Needle-style tests	Can the model locate a planted fact in long distractor text?	Quick sanity check for retrieval-from-context; too shallow for deep reasoning.
LongBench / LongBench v2	Can the model handle realistic long documents across tasks?	Broad long-context understanding; v2 raises reasoning difficulty.
RULER / ∞ Bench	Does performance hold as sequence length and complexity grow?	Stress test for <i>effective</i> context size, not just accepted length.
Truthfulness / hallucination sets	Does the model avoid unsupported or false claims?	Useful when factuality matters beyond retrieval.
Citation benchmarks	Can the model attach precise support spans to answer sentences?	Needed when verifiability matters, not only answer text.

Bottom line: No single benchmark settles the whole problem; long-context use, truthfulness, and citation quality must be evaluated separately.

Five historical bottlenecks

Family	Core question	Representative works
Dense attention	How should tokens share information inside one context window?	Transformer (2017)
Memory extension	How can the model keep older information without rereading everything?	Transformer-XL, Compressive Transformer
Sparse efficiency	How can we avoid $O(L^2)$ attention cost?	Reformer, Longformer, BigBird
Position handling	How can attention scores stay meaningful at unseen positions?	ALiBi, RoPE
External evidence	How can the model use retrieved evidence instead of only parametric memory?	REALM, RAG, FiD, RETRO, Memorizing Transformers

Why we organize history this way

The source survey groups papers by *bottleneck*, not by year, because later methods inherit different ideas from different branches.

Dense-attention root: the Transformer

- The Transformer made full self-attention the central architecture for sequence modeling.
- Historically, this is the object that later long-context work keeps modifying.

Matrix attention

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

Symbols: Q, K, V = batched query, key, value matrices; M = causal or sparse mask; L = sequence length.

Interpretation: Every position can compare itself to every relevant earlier position and copy a weighted mixture of their values.

Why it matters: The cost of forming $QK^\top$ is quadratic in L , which creates the first major long-context bottleneck.

Bottom line: Later methods usually change one of three things: which positions interact, how positions are encoded, or what memory keys and values are available.

Memory extension: Transformer-XL and Compressive Transformer

Transformer-XL

$$\tilde{H}_{\tau}^{(r-1)} = [\text{mem}_{\tau-1}^{(r-1)}; H_{\tau}^{(r-1)}]$$

Symbols: $H_{\tau}^{(r-1)}$ = layer- $(r-1)$ states for current segment τ ; $\text{mem}_{\tau-1}^{(r-1)}$ = cached states from the previous segment; $[\cdot; \cdot]$ = concatenation.

Interpretation: Current tokens can attend to both the current segment and a cached memory from earlier segments.

Why it matters: This showed that attention context can extend by reusing hidden states rather than rereading everything.

Compressive Transformer

$$C_t = [f_c(\overline{M}_t); C_{t-1}^{\text{trim}}]$$

Symbols: $\overline{M}_t$ = old memory to compress; f_c = compression operator; C_t = compressed long-term memory.

Interpretation: Recent memory is kept in high resolution, while older memory is compressed instead of discarded.

Why it matters: This added a second timescale of memory and foreshadowed later compressive long-range memory mechanisms.

Family-level takeaway

Transformer-XL adds **segment recurrence**; Compressive Transformer adds **older compressed memory**. Both attack the same bottleneck: how to keep useful information beyond one segment.

Sparse efficiency: Reformer, Longformer, BigBird

Method family	Core mechanism	Main lesson
Reformer	Approximate nearest-neighbor attention via locality-sensitive hashing (LSH) buckets	Not every token pair needs to be compared directly.
Longformer	Local sliding-window attention plus a small set of global tokens	Long documents often have local structure with a few globally important positions.
BigBird	Local + global + random sparse edges	Sparse attention can still permit effective information flow across a long sequence.

What this family does *not* solve

These methods primarily reduce **cost**. They do not by themselves solve the later out-of-distribution positional problem that dominates RoPE-scaling work.

Sparse efficiency: core mechanisms

Notation: $\mathcal{A}(i)$ = attention set of token i ; LSH = locality-sensitive hashing; $W(i)$ = local window; G = global tokens; $R(i)$ = random token set.

Reformer

Bucket tokens by content, then attend mostly within the same bucket:

$$b(i) = \text{LSH}(x_i), \quad \mathcal{A}(i) = \{j : b(j) = b(i)\}$$

LSH hashes random projections of x_i , so similar token vectors are likely to land in the same bucket.

Longformer

Mix local attention with a small set of global tokens:

$$\mathcal{A}(i) = W(i) \cup G$$

The window handles nearby context; G is usually a small predesignated set such as CLS, question, or task marker tokens.

BigBird

Keep local and global edges, then add random links:

$$\mathcal{A}(i) = W(i) \cup G \cup R(i)$$

Here $R(i)$ is a small random sample of links for token i (or its block); these random edges shorten graph distance.

Position handling: ALiBi versus RoPE

ALiBi

$$s_{ij}^{(h)} = \frac{q_i^\top k_j}{\sqrt{d_k}} - m_h(i - j), \quad j \leq i$$

Symbols: m_h = head-specific slope.

Interpretation: Farther tokens receive a linear distance penalty.

Why it matters: The rule extrapolates to unseen distances without learned lookup tables.

RoPE

$$\tilde{q}_m = R(m\theta)q, \quad \tilde{k}_n = R(n\theta)k$$

$$\tilde{q}_m^\top \tilde{k}_n = q^\top R((n - m)\theta)k$$

Symbols: $R(\cdot)$ = 2D rotation matrix; θ = frequency.

Interpretation: Relative position appears as relative phase inside the inner product.

Why it matters: RoPE became the main base mechanism later extended by PI, YaRN, and LongRoPE-style methods.

RoPE: why rotation gives relative position

One 2D coordinate pair

$$R(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}$$

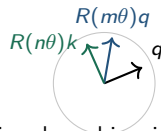
$$\tilde{q}_m = R(m\theta)q, \quad \tilde{k}_n = R(n\theta)k$$

$$\begin{aligned} \tilde{q}_m^\top \tilde{k}_n &= q^\top R(m\theta)^\top R(n\theta)k \\ &= q^\top R(-m\theta)R(n\theta)k \\ &= q^\top R((n-m)\theta)k \end{aligned}$$

Symbols: m, n = positions; θ = band frequency; $R(\cdot)$ = rotation matrix. One band = one 2D coordinate pair.

How to read it

- Frequency is a property of the RoPE band j , not of the token.
- Token m rotates its query by angle $m\theta$.
- Token n rotates its key by angle $n\theta$.
- The score depends only on the angle gap $(n-m)\theta$.
- If RoPE rotates all d_{head} coordinates, one head uses $d_{\text{head}}/2$ bands.
- Large unseen angle gaps motivate PI-, YaRN-, and LongRoPE-style fixes.



relative phase drives interaction

External evidence as memory

- This branch treats retrieved evidence as a first-class ingredient of prediction rather than hoping the model already memorized everything.

Latent-document view

$$p(y \mid x) = \sum_{z \in \mathcal{D}} p_{\eta}(z \mid x) p_{\theta}(y \mid x, z)$$

Symbols: x = query; z = retrieved document or passage; η = retriever parameters; θ = reader/generator parameters.

Interpretation: Prediction depends on a hidden choice of supporting evidence.

Why it matters: REALM and RAG made retrieval a modeling choice, FiD improved evidence fusion, and RETRO / Memorizing Transformers moved retrieval closer to the LM loop.

Historical synthesis

Framework A mainly descends from the **position-handling** branch; Framework B mainly descends from the **external-evidence** branch, with memory and efficiency ideas as supporting threads.

Why RoPE-based models break at very long length

- In RoPE, position enters as a rotation angle proportional to the token index.
- If test-time positions are far beyond the training range, the model sees unfamiliar relative phases.

Failure diagnosis

$$\tilde{q}_m^\top \tilde{k}_n = q^\top R((n - m)\theta)k$$

Symbols: m, n = positions; θ = frequency; $R((n - m)\theta)$ = relative rotation determined by position gap.

Interpretation: Attention scores depend on relative phase. Very large unseen gaps create phase patterns the model may never have learned to use stably.

Why it matters: The long-context problem is not only memory size; it is also out-of-distribution position geometry.

Bottom line: A long advertised window does not guarantee stable use of far-away evidence.

A unifying view: remap positions before attention

- Many modern long-context methods can be written as **position remapping**.
- Instead of feeding raw position m , feed a transformed position $g(m)$.

Generic design goal

choose g so that $g(m) \approx m$ on short contexts, but compresses or regularizes large m

Symbols: g = position-remapping function.

Interpretation: Keep short-range geometry familiar while shrinking dangerous long-range positional gaps into a safer regime.

Why it matters: This single idea unifies PI, YaRN, LongRoPE, and several adaptation recipes.

Bottom line: Framework A is best understood as a geometry-preservation problem, not as a bag of unrelated tricks.

PI: the canonical first modern recipe

- Positional Interpolation (PI) says: do not extrapolate raw positions far beyond training range.
- Instead, linearly compress test-time positions back into the familiar range before applying RoPE.

PI map

$$g_{\text{PI}}(m) = m \cdot \frac{L_{\text{train}}}{L_{\text{test}}}$$

Symbols: m = raw test-time position; L_{train} = training context length; L_{test} = target context length.

Interpretation: All relative phases are shrunk by the factor $L_{\text{train}}/L_{\text{test}}$.

Why it matters: Large unseen offsets in raw position space become more familiar offsets in the rotated coordinate space, which makes extension much stabler than raw extrapolation.

Why PI mattered

It gave the community a simple baseline that often works with only modest continued training.

YaRN: refine interpolation, do not abandon it

- PI uses one global compression factor.
- YaRN observes that different distance regimes and frequency bands may need different treatment.

Abstract YaRN view

$$g_{\text{YaRN}}(m; \theta_j) = \alpha_j(m) m$$

Symbols: θ_j = RoPE frequency band; $\alpha_j(m)$ = shrinkage schedule that may vary by band or position range.

Interpretation: High-frequency coordinates are more sensitive to large positions, so they may need stronger correction than low-frequency coordinates.

Why it matters: Frequency-aware remapping can preserve more short-context structure while taming the most unstable long-range phases.

Bottom line: YaRN is a more refined interpolation schedule, not a separate worldview from PI.

YaRN: how the shrinkage schedule is designed

Compared with PI

$$g_{\text{PI}}(m) = \beta m, \quad \beta = \frac{L_{\text{train}}}{L_{\text{test}}}$$

$$g_{\text{YaRN}}(m; \theta_j) = \alpha_j(m)m$$

PI uses one constant factor. YaRN lets shrinkage depend on both position m and frequency band j .

Design intuition

- $\alpha_j(m) \approx 1$ for short positions: preserve local behavior the model already knows.
- $\alpha_j(m) < 1$ for very long positions: compress dangerous phase growth.
- Higher-frequency RoPE bands use smaller $\alpha_j(m)$: they rotate faster and destabilize sooner.

Schematic view

$$\alpha_j(m) \approx \begin{cases} 1, & m \text{ small} \\ \text{ramp down,} & m \text{ medium} \\ \beta_j < 1, & m \text{ large} \end{cases} \quad \beta_j \text{ smaller for higher-frequency } j$$

Training-time support: PoSE, LongLoRA, LongAlign

Method	What it changes	Why it helps
PoSE	Training distribution over positions	Simulates large positional gaps inside shorter training windows.
LongLoRA	Adaptation cost	Uses sparse training-time attention and parameter-efficient updates for cheaper extension.
LongAlign	Long instruction tuning	Adds long instruction-following data, packing, and loss balancing so extended models still follow long tasks well.

How to read this family

These are not new positional mechanisms. They are **adaptation recipes** that make a chosen remapping strategy trainable and useful in practice.

LongRoPE and LongRoPE2

- LongRoPE searches for nonuniform remapping schedules.
- Extension is staged rather than done in one jump.
- LongRoPE2 makes short-context preservation explicit.

Two-regime objective

$$\min_{\psi, \vartheta} \mathcal{L}_{\text{long}}(\vartheta; g_{\psi}(m)) + \beta \mathcal{L}_{\text{short}}(\vartheta; m)$$

Symbols: g_{ψ} = searched remapping; ψ = remapping parameters searched or re-optimized across stages; ϑ = model weights fine-tuned by backprop; $\mathcal{L}_{\text{long}}$ = long-context objective; $\mathcal{L}_{\text{short}}$ = short-context preservation term; β = trade-off weight.

Interpretation: Extend long-context behavior without destroying the short-context competence the base model already had.

Why it matters: Million-scale context extension is possible only if the positional shift is staged and short-context regression is actively controlled.

Bottom line: A staged path such as 4k $\rightarrow$ 128k $\rightarrow$ 256k $\rightarrow$ 2M reduces positional shock at each step.

Complementary execution ideas

- Some recent methods change how long-range memory is **executed** rather than how RoPE itself is remapped.

Idea	Mechanism	Role
StreamingLLM	Keep a small set of attention-sink tokens plus a rolling local cache	Stabilizes streaming generation with bounded memory.
Infini-attention	Combine local attention with compressive memory	Revives the memory-extension branch in the LLM era.
ReAttention	Position-agnostic top- k preselection before ordinary attention	Offers a training-free route to much longer effective context.

Bottom line: Extreme context length can be helped by memory execution tricks even when the positional map itself is unchanged.

Complementary execution ideas: memory mechanisms

StreamingLLM

$$\mathcal{A}(t) = S \cup W_t$$

Symbols: S = attention-sink tokens kept for the whole stream; W_t = recent rolling local window. **Interpretation:** Each new token attends to a few persistent anchors plus recent context, rather than the full past. Sink tokens behave like stable anchors, so generation remains well-conditioned even after older tokens are evicted.

Infini-attention

$$y_t = \text{Attn}_{\text{local}}(x_{t-w:t}) + \text{Read}(M_{t-1}, q_t), \quad M_t = \text{Update}(M_{t-1}, k_t, v_t)$$

Symbols: M_t = compressive memory state; Read / Update = memory retrieval and write; w = local exact-attention window. **Interpretation:** Keep exact local attention for nearby tokens, but also maintain a compressed summary of older context. The memory grows by summary rather than by storing the full old KV cache.

Complementary execution ideas: ReAttention

Position-agnostic preselection

$$\mathcal{J}_i = \text{TopK}_j \text{ sim}(q_i, k_j), \quad \ell_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}} \text{ only for } j \in \mathcal{J}_i$$

Symbols: $\mathcal{J}_i$ = candidate set chosen before full attention. **Interpretation:** First do a position-agnostic top- k search for promising keys, then run ordinary position-aware attention only on those candidates. This reduces cost and weakens dependence on full-range positional extrapolation.

Why this is complementary to RoPE remapping

PI, YaRN, and LongRoPE try to make very large positional gaps safer. ReAttention instead avoids scoring every far-away token by narrowing the candidate set before ordinary attention is applied.

Bottom line: These methods do not mainly change the positional map; they change which memories are kept, compressed, or even considered at execution time.

Framework A: final algorithmic template

Generic remapped attention logit

$$\ell_{ij}^{(h)}(\psi) = \frac{\langle \text{RoPE}(q_i, g_\psi(m_i)), \text{RoPE}(k_j, g_\psi(m_j)) \rangle}{\sqrt{d_k}} + b_{ij}^{(h)}$$

Symbols: g_ψ = remapping function; m_i, m_j = raw positions; $b_{ij}^{(h)}$ = mask or extra bias.

Interpretation: Keep the same attention mechanism but apply RoPE at transformed positions rather than raw positions.

Why it matters: PI, YaRN, LongRoPE, and LongRoPE2 differ mainly in how g_ψ is chosen and how the model is adapted.

- 1 Diagnose the base model's trained length and positional mechanism.
- 2 Choose a remapping that preserves short-range geometry and compresses long-range geometry.
- 3 Pick a feasible adaptation recipe.
- 4 Preserve short-context quality explicitly.
- 5 Add memory tricks if the target window is extreme.

How Framework A is evaluated

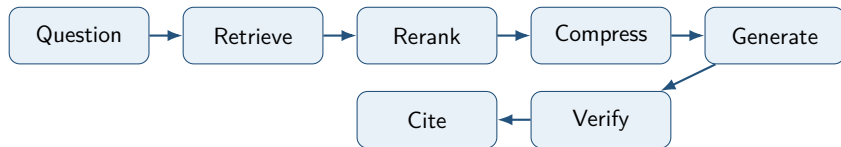
Work	What it changed	Why it matters
Lost in the Middle	Sweeps evidence location inside long prompts	Shows that middle-position evidence is often much harder than end-position evidence.
LongBench / v2	Broad long-context multi-task evaluation	Moves the field beyond passkey-style tests.
RULER	Stress tests effective context size	Shows nominal windows often overstate usable length.
∞ Bench	Pushes to 100k+ lengths with harder tasks	Exposes the gap between synthetic retrieval and realistic performance.

Evaluation warning

Measure sensitivity to **evidence position**, not only to **total length**. A model can look good on end-position cases while failing badly in the middle.

Grounded generation is a pipeline problem

- A faithful system must do more than decode an answer.
- It must decide whether retrieval is needed, which evidence matters, how to pack it, and how to verify and cite claims.



Bottom line: Hallucination reduction is not just a decoding trick; it is control over the whole evidence pipeline.

Latent evidence as a hidden choice

Latent-variable view

Let z denote the hidden evidence unit or evidence subset that actually supports the answer.

Exact latent view

$$p(y \mid x) = \sum_{z \in \mathcal{Z}} p_{\eta}(z \mid x) p_{\theta}(y \mid x, z)$$

Interpretation: Average over all candidate evidence units.

Practical top- k view

$$\hat{\mathcal{Z}}(x) = \text{TopK}_{z \in \mathcal{Z}} r_{\eta}(x, z)$$

$$p(y \mid x) \approx \sum_{z \in \hat{\mathcal{Z}}(x)} \bar{p}_{\eta}(z \mid x) p_{\theta}(y \mid x, z)$$

Interpretation: Keep only a small high-scoring subset and renormalize over it.

Read this as

The ideal model marginalizes over all plausible evidence units, while a real system keeps only a small retrieved set. If the right support never enters that retained set, the generator is already missing the evidence it needs.

Latent evidence: reading and failure modes

How to read this

Symbols: $\mathcal{Z}$ = all candidate evidence units; r_η = retrieval score; $\hat{\mathcal{Z}}(x)$ = retained top- k evidence set; $\bar{p}_\eta$ = retriever weights renormalized over the retained set. **Why it matters:** This is the bridge from a clean probabilistic view to real RAG-style systems with finite retrieval budgets.

Where hallucination enters

Either the right evidence never reaches $\hat{\mathcal{Z}}(x)$, or the generator produces claims not actually supported by the retained evidence.

Retrieval, reranking, and compression

Three subproblems

$$z_{1:k} = \text{TopK}_{z \in \mathcal{Z}} r_{\eta}(x, z), \quad \text{rerank with } \rho_{\phi}(x, z_i), \quad |c(z_{1:k})| \leq B$$

Symbols: r_{η} = retriever score; ρ_{ϕ} = stronger reranker score; $c(\cdot)$ = compressed evidence representation; B = prompt budget.

Interpretation: Retrieval aims for recall, reranking aims for precision, and compression aims to fit the prompt without dropping critical evidence.

Why it matters: Different system failures appear at different stages; a bad final answer may be caused by retrieval miss, reranking error, or unsafe compression.

Compression caveat

A summary can preserve the main rule but drop a tiny exception clause. This is one reason regulated settings often need explicit citation and exception handling.

From selected evidence to faithful answers

Faithfulness-regularized objective

$$\mathcal{L}(\theta, \eta) = \mathcal{L}_{\text{ans}} + \lambda \mathcal{L}_{\text{faith}} + \gamma \mathcal{L}_{\text{cite}}$$

Symbols: $\mathcal{L}_{\text{ans}}$ = answer quality term; $\mathcal{L}_{\text{faith}}$ = unsupported-claim penalty; $\mathcal{L}_{\text{cite}}$ = citation alignment term; λ, γ = weights.

Interpretation: A useful system should not optimize only answer text; it should also optimize support and attribution.

Why it matters: This objective unifies a large share of modern work: some papers change retrieval, some change verification, and some directly optimize citation.

Why retrieval alone is not enough

Even perfect retrieval cannot prevent unsupported paraphrase, overgeneralization, or wrong span selection during answer formation.

Verification and revision methods

Method	Mechanism	Best way to think about it
RARR	Research and revise model outputs with external evidence	Post-hoc factual repair loop.
CoVe	Explicit verification steps before finalizing the answer	Structured self-checking pipeline.
SelfCheckGPT	Compare multiple sampled generations for consistency	Black-box hallucination detector when labeled support is unavailable.

Family-level lesson

These methods intervene **after or during answer formation**. They do not mainly change the retriever; they change how unsupported content is detected or revised.

Adaptive retrieval and correction

Self-RAG

- Retrieval is not fixed in advance.
- The model can choose actions like retrieve, continue, or critique.
- Reflection tokens guide answer quality and evidence use.

CRAG

- Retrieval quality is explicitly estimated.
- Low-quality evidence is corrected before generation.
- The generator is protected from blindly conditioning on bad retrieval.

Key distinction

Self-RAG makes retrieval adaptive *inside* generation. CRAG makes retrieval robust *before* generation by checking evidence quality first.

Structured retrieval for corpus-level synthesis

When flat retrieval works

Local fact, one narrow evidence cluster, and a small set of passages is usually enough.

When structure helps

Corpus-wide questions depend on many passages at once, and a longer top- k list may still miss global themes and links.

GraphRAG mechanism

$$\mathcal{Z}_{\text{raw}} \rightarrow G = (V, E) \rightarrow \mathcal{Z}_{\text{summary}} \rightarrow y$$

Symbols: $G = (V, E)$ = graph built from corpus entities and relations; $\mathcal{Z}_{\text{summary}}$ = community-level summaries or partial answers.

What changes

Interpretation: The system first structures the corpus, then retrieves communities or summaries rather than only raw passages. **Why it matters:** This helps when the answer requires corpus-wide theme synthesis, not just one local fact.

Other paths

LongRAG packs more context; R²AG injects retrieval-side signals into generation.

Citation as a first-class objective

LongCite

- Direct supervision for answer sentences paired with supporting spans.
- Treats citation generation as a trainable target, not a formatting afterthought.

SelfCite

- Uses necessity and sufficiency checks through context ablation.
- Learns better citations without requiring all labels to be written by humans.

SelfCite-style reward

$$S_{\text{cite}}(c, z) = \underbrace{[p(c | z) - p(c | \emptyset)]}_{\text{sufficiency}} + \underbrace{[p(c | E) - p(c | E \setminus z)]}_{\text{necessity}}$$

Symbols: c = claim; z = candidate citation span; E = full evidence.

Interpretation: A good citation span should be enough to support the claim and important enough that removing it hurts the claim.

Why it matters: Citation becomes an optimization target rather than a presentation detail.

Framework B: final algorithmic template

- 1 Choose evidence granularity and retrieve high-recall candidates.
- 2 Rerank or correct the candidate set.
- 3 Compress or structure evidence to fit the budget safely.
- 4 Generate from evidence rather than from parametric memory alone.
- 5 Verify and cite at the claim or sentence level.

Core generation step

$$y \sim p_{\theta}(y \mid x, c(\hat{z}_{1:k}))$$

Symbols: $\hat{z}_{1:k}$ = refined evidence set after retrieval, reranking, or correction; $c(\cdot)$ = packed or structured representation.

Interpretation: Answer generation should be downstream of evidence control, not independent from it.

Why it matters: Most papers in Framework B differ mainly in *where* they intervene in this pipeline.

How Framework B is evaluated

Benchmark	Main target	Why it matters
TruthfulQA	Avoiding common falsehoods	Separates truthfulness from generic fluency.
HaluEval	Hallucination in instruction-following settings	Provides a broad stress test for unsupported generation.
FActScore	Atomic-claim factual precision in long-form text	Encourages claim-level evaluation instead of whole-answer scoring.
LongBench-Cite / LongCite	Fine-grained support spans in long-context QA	Pushes citation quality into mainstream evaluation.

Bottom line: Framework B is broader than “RAG”: it includes retrieval, reranking, compression, critique, correction, graph structuring, and citation alignment.

When to use long context, retrieval, or both

Situation	Best default	Reason
Single long document; answer depends on many nearby parts	Long context	The evidence is already inside one source; retrieval may add unnecessary machinery.
Huge corpus; answer depends on a few tiny passages	Retrieval or hybrid	Reading everything is wasteful; selection is the main problem.
Question requires corpus-wide synthesis	GraphRAG-style structure or hybrid	Flat retrieval often misses global themes and relationships.
Regulated or high-stakes setting	Hybrid with citation	Verifiability matters as much as answer text.
Open-weight deployment with limited hardware	Recipe-based extension + selective retrieval	Full long-context pretraining may be too expensive.

Bottom line: Choose by *evidence shape*: one long source, sparse passages, global synthesis, or auditable reasoning.

Failure-analysis checklist

Diagnose failures in this order

- 1 Was the right evidence present in the raw corpus at all?
- 2 Did retrieval return it?
- 3 Did reranking or compression discard it?
- 4 If it reached the prompt, where was it positioned?
- 5 Did the model answer from the evidence, or from unsupported parametric memory?
- 6 Was the answer right but the citation wrong?

Why this checklist matters

It prevents the common mistake of blaming the generator for an upstream retrieval or packing error.

What recent work changed in practice

- On the **Framework A** side, simple RoPE remapping is now the baseline; the real differentiators are adaptation quality and evaluation by evidence position.
- On the **Framework B** side, retrieval systems increasingly add correction, structure, and citation objectives rather than stopping at top- k passage lookup.
- For deployment, the dominant pattern is a **hybrid**: enough window for local coherence, targeted retrieval for sparse evidence, and explicit verification/citation for auditable outputs.

Running-example implication

For the policy-manual task, do not choose between “just make the window bigger” and “just use RAG.” Use enough window for nearby context, retrieval for the right sections, and citations for verification.

What remains genuinely hard

- **True million-scale reasoning:** reading a million tokens is easier than reasoning correctly over them.
- **Exception preservation under compression:** tiny exception clauses are easy to lose.
- **Reliable claim decomposition:** citation and factual evaluation depend on meaningful atomic claims.
- **Joint optimization of short and long contexts:** extension methods can still hurt short-range quality.
- **Robust evaluation:** synthetic tests are useful, but cannot replace realistic long-context reasoning and attribution tasks.

Bottom line: The frontier is no longer “can the model fit the tokens?” It is “can the system reason faithfully and verifiably over them?”

Six ideas to remember

- ① A large context window is not the same as long-context skill.
- ② Most modern long-context extension can be understood as **position remapping plus careful adaptation**.
- ③ Retrieval and long context solve different failure modes.
- ④ Hallucination reduction is a **pipeline problem**: retrieve, rerank, compress, generate, verify, cite.
- ⑤ Citation is a modeling target when answers must be auditable.
- ⑥ The strongest practical systems usually combine stable position handling with explicit evidence control.

Final takeaway

Stabilize the model's use of long positions *inside* the model, and control evidence *around* the model.

Notation summary

Symbol	Meaning
w_t	token ID at position t
x_t	embedding vector of token w_t
Q, K, V	query, key, and value matrices or vectors
M	causal or sparse attention mask
ℓ	input length in tokens
π	position of answer-bearing evidence inside the input
$\mathcal{M}(f; \ell, \pi)$	task metric as a function of length and evidence position
$\text{supp}(c, E)$	indicator that claim c is supported by evidence E
z	latent evidence variable: passage, chunk, graph summary, or citation span
$g(m)$	position-remapping function for long-context extension
B	prompt-budget constraint for evidence packing

Method clustering map

Cluster	Main distinction that matters	Representative methods
RoPE remapping	How raw positions are transformed before RoPE	PI, YaRN, LongRoPE, LongRoPE2
Adaptation recipes	How the model is trained after choosing a target long regime	PoSE, LongLoRA, LongAlign
Execution / memory	How very long context is executed with bounded compute or memory	StreamingLLM, Infini-attention, ReAttention
Evidence control	Which evidence enters the model and in what order	RAG, FiD, Self-RAG, CRAG
Structured retrieval	Whether raw passages are enough or a higher-level representation is needed	GraphRAG, LongRAG, R ² AG
Citation optimization	Whether citation is supervised or self-supervised	LongCite, SelfCite

Selected references mentioned in the deck

Historical roots

Transformer (2017)
Transformer-XL (2019)
Compressive Transformer (2020)
Reformer (2020)
Longformer (2020)
BigBird (2020)
REALM (2020)
RAG (2020)
FiD (2021)
ALiBi (2022)
RoPE / RoFormer (2021)
RETRO (2022)
Memorizing Transformers (2022)

Modern anchors

PI (2023)
YaRN (2023)
Lost in the Middle (2024)
LongBench / LongBench v2 (2023–2025)
LongLoRA (2023)
LongRoPE / LongRoPE2 (2024–2025)
Self-RAG (2024)
CRAG (2024)
GraphRAG (2024)
LongRAG (2024)
LongCite (2024)
SelfCite (2025)

Canonical papers as anchor points

Work	Framework	Main move	Why it changed practice
PI	A	Linear position remapping	Gave a simple baseline for RoPE extension.
YaRN	A	More refined interpolation plus efficient adaptation	Became a practical improvement over plain PI.
Lost in the Middle	A eval	Probe evidence location inside long prompts	Changed evaluation norms: position matters.
Retrieval meets Long Context LLMs	A+B bridge	Compare long windows, retrieval, and hybrids	Reframed a false long-context-versus-RAG dichotomy.
LongLoRA	A	Cheap long-context adaptation for open models	Lowered the barrier to extension in practice.
LongRoPE	A	Searched nonuniform remapping and staged extension	Marked the million-scale context milestone.
Self-RAG	B	Retrieval control and critique inside generation	Made retrieval adaptive instead of static.
GraphRAG	B	Graph/community structure for global questions	Popularized structured retrieval beyond flat passage search.
LongCite	B	Direct supervision for fine-grained citation	Made citation a primary trainable target.

Framework A support methods at a glance

Method	What changes	Best mental model
PoSE	Training distribution over positions	Simulate large positional gaps inside shorter windows by assigning offset position ranges to chunks.
LongLoRA	Adaptation cost	Use shifted sparse attention during training and low-rank updates so long-context tuning fits limited hardware.
LongAlign	Long instruction alignment	Add long instruction data, packing, batching, and loss balancing so the extended model still follows long tasks well.
StreamingLLM	Streaming-time memory execution	Keep a small set of anchor tokens plus a rolling local cache during long streaming inference.
Infini-attention	Compressive memory channel	Combine local attention with a compressed memory that can grow beyond a fixed global window.
ReAttention	Training-free preselection	Select a position-agnostic top- k subset before ordinary position-aware attention.

Bottom line: These methods support Framework A in three distinct ways: training distribution, adaptation efficiency, and inference-time memory execution.

Historical external-evidence lineage

Work	Main move	Lasting lesson
REALM	Retrieved documents as latent variables during pre-training	Retrieval can be part of the model, not only a wrapper around it.
RAG	Canonical retrieve-then-generate latent-document formulation	Established the baseline probabilistic view for evidence-conditioned generation.
FiD	Encode passages separately, fuse them at the decoder	Strong evidence fusion can matter as much as retrieval itself.
RETRO	Chunk-level retrieval inside the LM loop	Retrieval can sit closer to token prediction, not only before final answer generation.
Memorizing Trans-formers	Nearest-neighbor memory attached to attention	External memory can behave like a nonparametric extension of model knowledge.

Bottom line: This historical branch leads directly into Framework B: select and fuse evidence explicitly rather than relying only on parametric memory.

Recent works: canon versus frontier

Canon

PI, YaRN, Lost in the Middle, Retrieval meets Long Context LLMs, LongLoRA, Self-RAG, GraphRAG, LongCite, LongRoPE

Frontier

PoSE, LongAlign, ReAttention, LongRoPE2, UltraLong, CRAG, SelfCite

Selection rule

Canon changes the field's default framing, mechanism, or evaluation culture. **Frontier** is promising and influential, but its long-term canonical status is still maturing.

Bottom line: This appendix slide is a lookup aid, not a cue to expand the main deck into one-paper-per-slide coverage.